

Graph-Based Recommendation & Community Detection Engine

A from-scratch graph analytics engine implementing PageRank, community detection, and friend recommendation on a synthetic social network graph.

Course Foundations

This project applies **Combinatorics & Graph Theory** (graph structures, connectivity, eigenvector-based centrality), **Design and Analysis of Algorithms** (BFS complexity analysis, Union-Find with path compression), and **Big Data Analysis** (designing algorithms that scale to large graphs).

System Overview

A synthetic social network is generated using preferential attachment (Barabasi-Albert style growth), which produces realistic degree distributions similar to real social graphs — a few highly-connected hub nodes and many low-degree nodes. On top of this graph, four algorithms are implemented entirely from scratch (no external graph libraries):

1. Degrees of Separation — BFS Shortest Path

Standard breadth-first search finds the shortest connection path between any two users, with $O(V + E)$ time complexity. This answers questions like "how many connections separate user A from user B?"

2. Influence Ranking — PageRank via Power Iteration

PageRank models influence as a stationary distribution of a random walk on the graph, computed via iterative power-method updates until convergence. Mathematically, this approximates the principal eigenvector of the graph's (damped) transition matrix — connecting directly to eigenvalue concepts from linear algebra / stochastic processes.

```
def pagerank(adj, num_nodes, damping=0.85, iterations=50, tol=1e-8):
    N = num_nodes
    rank = {n: 1.0 / N for n in range(N)}
    out_degree = {n: max(1, len(adj[n])) for n in range(N)}
    for it in range(iterations):
        new_rank = {n: (1 - damping) / N for n in range(N)}
        for u in range(N):
            share = rank[u] / out_degree[u]
            for v in adj[u]:
                new_rank[v] += damping * share
        diff = sum(abs(new_rank[n] - rank[n]) for n in range(N))
        rank = new_rank
        if diff < tol:
            break
    return rank
```

3. Community Detection — Union-Find on Similarity Graph

Pairwise Jaccard similarity ($|shared\ neighbors| / |union\ of\ neighbors|$) is computed for connected node pairs. Pairs exceeding a similarity threshold are merged using a Union-Find (Disjoint Set Union) structure with path compression and union by rank, giving near- $O(1)$ amortized merge/find operations. Connected components in this similarity graph form the detected communities.

4. Friend Recommendation — "People You May Know"

For a given user, candidate recommendations are drawn from friends-of-friends (2-hop neighborhood) and ranked by Jaccard similarity to the target user's own neighborhood — the same principle used in real "People You May Know" features.

Results (measured on a 5,000-node, ~20,000-edge synthetic graph)

Algorithm	Result	Time
BFS shortest path (node 0 -> node 4999)	3 degrees of separation	1.25 ms
PageRank (50 iterations, 5000 nodes)	Top node: score 0.00478, degree 225	179 ms
Community detection (threshold=0.03)	4,606 communities, largest size 27	33 ms
Friend recommendation (node 10, top-5)	Best match: node 13 (similarity 0.053)	<1 ms

All timings were measured directly on the synthetic graph described above; the graph itself was generated in 20–37 ms. Community detection reveals meaningful clustering structure beyond simple connectivity, since the full graph is single-component but splits into 4,606 tighter-knit communities under the similarity threshold.

Tech Stack

Python 3, standard library only (collections.deque, no external graph libraries), adjacency-set representation for O(1) neighbor lookups.

Possible Extensions

Weighted PageRank with edge weights, hierarchical (Louvain-style) community detection for better modularity, and migrating the adjacency structure to a distributed framework (e.g. Spark GraphX) to test true Big-Data-scale graphs (1M+ nodes).